# Validated Intent Compilation for Constrained Routing in LEO Mega-Constellations

[Author Names]
[Affiliation]
City, Country
ail

***Abstract*—Operating LEO mega-constellations requires translating high-level operator intents ("reroute financial traffic away from polar links under 80 ms") into low-level routing constraints—a task that demands both natural language understanding and network-domain expertise. We present an end-to-end system comprising three components: (1) a GNN cost-to-go router that distills Dijkstra-quality routing into a 152K-parameter graph attention network achieving 99.8% packet delivery ratio with 17× inference speedup; (2) an LLM intent compiler that converts natural language to a typed constraint intermediate representation using few-shot prompting with a verifier-feedback repair loop, achieving 98.4% compilation rate and 87.6% full semantic match on feasible intents in a 240-intent benchmark (193 feasible, 47 infeasible); and (3) an 8-pass deterministic validator with constructive feasibility certification that achieves 0% unsafe acceptance on all 47 infeasible intents (30 labeled + 17 discovered by Pass 8), with 100% corruption detection across 240 adversarial tests. End-to-end evaluation across five constrained routing scenarios confirms zero constraint violations with both routers. We further demonstrate that apparent performance gaps in polar-avoidance scenarios are entirely explained by topological reachability ceilings rather than routing quality, and that the LLM compiler outperforms a rule-based baseline by 29.5 percentage points on compositional intents. Our system bridges the semantic gap between operator intent and network configuration while maintaining the safety guarantees required for production deployment.**

***Index Terms*—LEO satellite networks, intent-based networking, graph neural networks, large language models, constrained routing, formal verification**

## I. Introduction

Low Earth Orbit (LEO) mega-constellations such as Starlink, OneWeb, and Kuiper are transforming global connectivity by deploying thousands of satellites interconnected via inter-satellite links (ISLs). Operating these networks presents unique challenges: the topology changes continuously as satellites orbit, polar regions experience periodic link dropout, and operators must enforce complex routing constraints spanning latency guarantees, region avoidance, node maintenance, and traffic prioritization.

Today, translating operator intent into network configuration requires manual specification of routing policies—a process that is slow, error-prone, and does not scale to the dynamic nature of LEO constellations. Intent-based networking (IBN) [1] promises to bridge this gap by allowing operators to express high-level goals that are automatically compiled into network configurations. However, existing IBN approaches target terrestrial networks with relatively stable topologies and do not address the unique constraints of satellite mega-constellations.

We identify three key challenges in intent-driven LEO routing:

1) **Compositional intent understanding.** Operator intents combine multiple constraint types ("disable plane 7, avoid polar links above 75°, and cap utilization at 80%") that must be correctly decomposed and mapped to formal constraint representations. Rule-based parsers handle simple intents but degrade sharply on compositional ones (40% vs. 82% accuracy).
2) **Safety-critical verification.** In production networks, a single undetected constraint violation can cascade into service outages. The compiler's output must be formally verified before reaching the routing layer, yet verification must be fast enough for interactive use.
3) **Efficient constrained routing.** Applying constraints modifies the network topology (disabling nodes, removing edges), and the router must compute valid paths on the constrained graph in real time. Traditional shortest-path algorithms are correct but too slow for per-packet decisions at scale.

We address these challenges with a three-component system :

**GNN Cost-to-Go Router** (Section III-B). A 3-layer graph attention network (152K parameters) trained via supervised distillation from Dijkstra shortest paths. It achieves 99.8% packet delivery ratio (PDR) while providing 17× inference speedup, enabling real-time per-packet routing decisions.

**LLM Intent Compiler** (Section III-C). A Qwen3.5-9B language model with 6-shot prompting converts natural language intents to a typed ConstraintProgram intermediate representation. A verifier-feedback repair loop corrects compilation errors, achieving 98.4% compilation rate and 87.6% full semantic match on 193 feasible benchmark intents.

**Deterministic Validator** (Section III-D). An 8-pass verification pipeline checks schema validity, entity grounding, type

safety, value ranges, constraint conflicts, physical admissibility, and reachability. It achieves 100% detection on structural corruptions and guarantees that no malformed constraint program reaches the routing layer.

Our contributions are:

- A typed constraint IR (ConstraintProgram) that formally bridges natural language intents and topology-level routing constraints, with grounding semantics for 10 hard and 5 soft constraint types.
- An LLM-based intent compiler with verifier-feedback repair that outperforms rule-based parsing by 29.5pp on compositional intents and generalizes to out-of-distribution phrasings (81.8% accuracy, 4.4pp degradation).
- A reachability separation analysis showing that apparent routing performance gaps under polar constraints are entirely explained by topological reachability ceilings, not routing quality.
- End-to-end evaluation demonstrating zero constraint violations across five scenarios with both GNN and Dijkstra routers.

## II. Related Work

**LEO constellation routing.** Routing in LEO mega-constellations has been studied extensively. Snapshot-based approaches [2] precompute routes for discrete time intervals but cannot adapt to dynamic failures. Contact graph routing (CGR) [3] handles time-varying topologies but assumes deterministic contact schedules. Recent work applies deep reinforcement learning (DRL) to LEO routing [4], [5], but DRL agents require online training and struggle with credit assignment over large action spaces. Our GNN cost-to-go approach avoids these issues through offline supervised distillation, achieving Dijkstra-equivalent quality with 17× speedup.

**GNN-based network optimization.** Graph neural networks have shown promise for combinatorial network problems including traffic engineering [6], link scheduling [7], and routing [8]. RouteNet [9] models network performance but does not produce routing decisions. Our work differs by training a GNN to directly predict per-destination next-hop decisions via cost-to-go distillation, enabling deployment as a drop-in routing engine.

**Intent-based networking.** IBN aims to translate operator goals into network configurations [1], [11]. Existing systems use template matching [12], ontology-based parsing [13], or domain-specific languages [14]. Recent work explores LLMs for network intent translation [15], [16], but without formal verification of the compiled output. Our system combines LLM compilation with deterministic verification, ensuring that the semantic flexibility of LLMs does not compromise network safety.

**LLMs for network management.** Large language models are increasingly applied to network tasks [17], [18]: configuration generation [19], anomaly diagnosis [20], and policy translation [21]. However, most approaches trust LLM output directly or use only syntactic validation. Our 8-pass validator goes beyond syntax to check entity grounding, type safety, physical admissibility, and reachability—providing the verification depth required for safety-critical infrastructure.

## III. System Design

### A. Problem Formulation

We consider a Walker Delta constellation with $P$ orbital planes and $S$ satellites per plane, yielding $N = P \times S$ nodes. Each satellite maintains up to $K = 4$ ISL links (2 intra-plane, 2 inter-plane). The constellation graph $G_t = (V, E_t)$ evolves over time as satellite positions change and polar links experience periodic dropout above inclination $\iota$.

An operator intent $\ell$ is a natural language string expressing routing constraints. The system must:

1) *Compile*: $\ell \xrightarrow{\text{LLM}} \mathcal{P}$, mapping intent to a ConstraintProgram $\mathcal{P}$.
2) *Verify*: $\mathcal{P} \xrightarrow{\text{Validator}} \mathcal{P}^{\checkmark}$, ensuring structural and physical validity.
3) *Ground*: $(\mathcal{P}^{\checkmark}, G_t) \xrightarrow{\Gamma} (M_V, M_E, \mathbf{u}, \mathbf{d})$, producing topology masks and flow constraints.
4) *Route*: $(G_t \odot M, \mathbf{d}) \xrightarrow{\text{GNN}} \Pi$, computing constrained routing tables.

**Definition 1** (ConstraintProgram)**.** *A ConstraintProgram $\mathcal{P}$ is a typed intermediate representation that captures operator intent as a tuple:*

$$\mathcal{P} = \langle \mathcal{F}, \mathcal{H}, \mathcal{S}, \mathcal{E}, \omega, \pi \rangle \tag{1}$$

*where $\mathcal{F}$ is a set of flow selectors, $\mathcal{H}$ is a set of hard constraints, $\mathcal{S}$ is a set of soft constraints, $\mathcal{E}$ is a set of event conditions, $\omega$ is an objective weight vector, and $\pi \in \{\texttt{critical}, \texttt{high}, \texttt{medium}, \texttt{low}\}$ is the priority level.*

**Definition 2** (Flow Selector)**.** *A flow selector $f \in \mathcal{F}$ identifies a subset of traffic flows:*

$$f = \langle \tau, r_s, r_d, n_s, n_d, p_s, p_d \rangle \tag{2}$$

*where $\tau \in \mathcal{T}$ is a traffic class (e.g., `financial`, `emergency`), $r_s, r_d \in \mathcal{R} \cup \{\perp\}$ are source/destination regions, $n_s, n_d \in [0, N) \cup \{\perp\}$ are source/destination node IDs, and $p_s, p_d \in [0, P) \cup \{\perp\}$ are source/destination orbital planes.*

**Definition 3** (Hard Constraint)**.** *A hard constraint $h \in \mathcal{H}$ must be satisfied; violation renders the program infeasible:*

$$h = \langle t_h, \sigma, v, c \rangle \tag{3}$$

*where $t_h \in \mathcal{T}_H$ is the constraint type, $\sigma$ is the target specifier, $v$ is the constraint value, and $c \in \mathcal{E} \cup \{\perp\}$ is an optional event condition.*

*The hard constraint type set is:*

$$\begin{aligned}\mathcal{T}_H = \{&\texttt{disable_node}, \texttt{disable_plane}, \texttt{disable_edge},\\ &\texttt{avoid_region}, \texttt{avoid_latitude}, \texttt{reroute_away},\\ &\texttt{max_latency_ms}, \texttt{max_hops},\\ &\texttt{k_edge_disjoint}, \texttt{min_capacity_reserve}\}\end{aligned} \tag{4}$$

TABLE I
CONSTRAINTPROGRAM HARD CONSTRAINT TYPES AND THEIR GROUNDING SEMANTICS.

| Type | Target | Grounding |
|---|---|---|
| `disable_node` | `node:`$n$ | $M_V[n] \leftarrow 0$ |
| `disable_plane` | `plane:`$p$ | $M_V[pS{:}(p{+}1)S] \leftarrow 0$ |
| `avoid_latitude` | `edges:ALL` | $M_E[(u,v)] \leftarrow 0$ if $\lvert\phi\rvert > \theta$ |
| `avoid_region` | `region:`$r$ | $M_E[(u,v)] \leftarrow 0$ if $u, v \in \mathcal{N}_r$ |
| `reroute_away` | `node:`$n$ | $M_V[n] \leftarrow 0$ (transit) |
| `max_latency_ms` | `flow_sel:`$i$ | $\mathbf{d}[f_i] \leftarrow v$ |
| `max_hops` | `flow_sel:`$i$ | hop limit on path |

**Definition 4** (Constraint Grounding)**.** *Given a constellation graph $G = (V, E)$ with $|V| = N$ nodes and topology state at time $t$, the grounding function $\Gamma$ maps a ConstraintProgram to topology modifications:*

$$\Gamma(\mathcal{P}, G, t) = \langle M_V, M_E, \mathbf{u}, \mathbf{d} \rangle \tag{5}$$

*where $M_V \in \{0,1\}^N$ is a node mask, $M_E \in \{0,1\}^{|E|}$ is an edge mask, $\mathbf{u} \in [0,1]^{|E|}$ is a per-edge utilization cap vector, and $\mathbf{d} : \mathcal{F} \to \mathbb{R}^+$ maps flow selectors to deadline values.*

*Grounding rules for topology-modifying constraints:*

- `disable_node`$(n)$*:* $M_V[n] \leftarrow 0$*, propagate to incident edges*
- `disable_plane`$(p)$*:* $\forall s \in [0, S) : M_V[p \cdot S + s] \leftarrow 0$
- `avoid_latitude`$(\theta)$*:* $\forall (u,v) \in E : |\phi_u| > \theta \vee |\phi_v| > \theta \Rightarrow M_E[(u,v)] \leftarrow 0$
- `avoid_region`$(r)$*:* $\forall (u,v) \in E : u \in \mathcal{N}_r \vee v \in \mathcal{N}_r \Rightarrow M_E[(u,v)] \leftarrow 0$

*where $\phi_n$ is the latitude of node $n$ and $\mathcal{N}_r$ is the set of nodes within region $r$.*

### B. GNN Cost-to-Go Router

The routing component must compute next-hop decisions for all origin-destination pairs on the (possibly constrained) topology graph. We train a GNN to approximate Dijkstra's cost-to-go function via supervised distillation.

*1) Architecture:* The encoder is a 3-layer Graph Attention Network (GAT) [10] with 128-dimensional hidden states and 4 attention heads per layer. Input node features $\mathbf{x}_i \in \mathbb{R}^{10}$ encode: satellite position (latitude, longitude, altitude), orbital parameters (plane ID, slot ID as sinusoidal encodings), and local topology statistics (degree, mean neighbor delay).

For each destination $d$, the scorer computes a cost-to-go estimate for each neighbor $j$ of node $i$:

$$\hat{c}(i, j, d) = \mathrm{MLP}\big([\mathbf{h}_i \| \mathbf{h}_j \| \mathbf{h}_d \| \mathbf{e}_{ij}]\big) \tag{6}$$

where $\mathbf{h}_i$ is the GAT embedding of node $i$, $\|$ denotes concatenation, and $\mathbf{e}_{ij}$ encodes edge features (delay, capacity). The next hop is $\arg\min_j \hat{c}(i, j, d)$.

*2) Training:* We generate 500 topology snapshots by sampling constellation states at random orbital phases. For each snapshot, Dijkstra's algorithm computes the optimal next-hop table $\Pi^* \in [K]^{N \times N}$. The model is trained for 200 epochs with cross-entropy loss on next-hop predictions, using a phased curriculum: 50 epochs on easy pairs (hop distance $\leq 5$), then 150 epochs on all pairs.

*3) Constrained Routing:* When constraints are active, the grounding function $\Gamma$ produces node mask $M_V$ and edge mask $M_E$. The constrained graph $G' = (V \odot M_V, E \odot M_E)$ is passed to the GNN, which computes routing tables on the reduced topology. This approach requires no retraining—the GNN generalizes to unseen topologies through its message-passing architecture.

### C. LLM Intent Compiler

The compiler translates natural language intents to ConstraintProgram JSON using a three-stage pipeline: few-shot prompting, JSON extraction, and verifier-feedback repair.

*1) Prompt Design:* The system prompt (approximately 800 tokens) specifies the constellation parameters, the complete ConstraintProgram JSON schema, all valid enum values, target format conventions, and 6 compilation rules. Six in-context examples cover single constraints (node disable, latency SLA), compositional constraints (plane disable + polar avoidance + utilization cap), and conditional constraints (event-triggered reroute).

*2) Repair Loop:* When the verifier rejects a compiled program, the error messages are appended to the conversation as a repair prompt. The compiler retries up to $k = 3$ times, with each attempt receiving the accumulated error context. This closed-loop design converts verifier precision into compiler accuracy: 89.2% of intents succeed on the first attempt, and the repair loop recovers an additional 8.7%.

*3) JSON Extraction:* The extractor handles three response formats: raw JSON, markdown-fenced JSON, and JSON embedded in explanatory text. It also strips reasoning tags (`<think>`) produced by instruction-tuned models. Robust extraction is critical because even high-quality LLMs occasionally wrap JSON in commentary.

### D. Deterministic Validation Pipeline

A key design principle of our system is that the LLM compiler operates *offline* and its output is never trusted directly. Every ConstraintProgram passes through an 8-pass deterministic validator before reaching the routing layer. This design is motivated by two observations: (1) LLMs can produce syntactically valid but semantically incorrect constraint programs, and (2) in safety-critical network infrastructure, a single undetected constraint violation can cascade into service outages.

The validator implements the following passes in sequence, with early termination on fatal errors:

1) **Schema Validation.** Checks structural completeness: all required fields present, valid priority levels, non-empty constraint types and targets. Rejects malformed programs before deeper analysis.
2) **Entity Grounding.** Verifies that all referenced entities exist in the constellation model: node IDs $\in [0, N)$, plane IDs $\in [0, P)$, region names $\in \mathcal{R}$, traffic classes $\in \mathcal{T}$. This

catches hallucinated entities (e.g., node 454 in a 400-node constellation).

3) **Type Safety.** Ensures constraints attach to semantically correct entity types: `max_latency_ms` must target a `flow_selector`, `disable_node` must target a `node`, `avoid_latitude` must target `edges`. Prevents type confusion errors where the LLM assigns a constraint to the wrong entity class.
4) **Value Range Checking.** Validates numeric parameters: latitudes $\in [-90, 90]$, latency values $> 0$, utilization caps $\in (0, 1]$, node IDs within constellation bounds. Catches out-of-range values that would produce undefined behavior.
5) **Conflict Detection.** Identifies contradictory constraints within the same program: a node cannot be simultaneously disabled and used as a routing waypoint; conflicting latency bounds on the same flow are flagged. Contradictions are promoted to errors (not warnings), ensuring logically inconsistent programs are rejected.
6) **Physical Admissibility.** Checks whether the constrained topology is physically realizable: latency deadlines below the single-hop physical minimum ($< 2.0$ ms) are rejected; latitude avoidance thresholds that remove $> 50\%$ of edges trigger warnings.
7) **Reachability Analysis.** Performs BFS on the constrained graph $G' = (V \odot M_V, E \odot M_E)$ to verify connectivity. Severe capacity loss ($\geq 75\%$ of nodes disabled) is rejected as non-functional; moderate loss ($\geq 50\%$) triggers warnings.
8) **Feasibility Certification.** For each demanded flow, constructs a routing witness on the constrained topology to certify that all hard constraints can be simultaneously satisfied. Five certified fragments cover the constraint space:
   - **F1** (topology only): BFS reachability
   - **F2** (+ latency): Dijkstra with deadline
   - **F3** (+ hops): BFS with hop limit
   - **F4** (+ latency + hops): hop-layered Dijkstra
   - **F5** (+ $k$-disjoint): Edmonds-Karp max-flow

   Three outcomes: ACCEPT (witness found), REJECT (no feasible routing exists), or ABSTAIN (unsupported constraint combination). Programs are rejected only on REJECT; ABSTAIN preserves soundness by deferring to the routing layer.

**Theorem 1** (Acceptance Soundness)**.** *If the feasibility certifier accepts a constraint program $P$ with witness $W$, then there exists a routing assignment satisfying all hard constraints of $P$ on the constrained topology.*

*Proof sketch.* By case analysis over fragments F1–F5. Each fragment's algorithm is a standard shortest-path or max-flow algorithm whose correctness is well-established. The witness $W$ is the concrete path (or path set) returned by the algorithm, which by construction satisfies the topology constraints (disabled nodes/edges excluded from the search graph), latency bounds (Dijkstra optimality), hop limits (BFS layering), and disjointness requirements (augmenting path decomposition). Unsupported combinations produce ABSTAIN, never ACCEPT, so no false acceptance is possible within the certified fragment space. □

**Design rationale.** We chose deterministic validation over learned or probabilistic checking for three reasons:

- *Completeness*: every structural error class is covered by at least one pass, achieving 100% detection on our corruption benchmark (8 error types $\times$ 30 injections = 240 tests).
- *Transparency*: each rejection includes a human-readable error message identifying the specific violation, enabling the repair loop to provide targeted feedback to the LLM.
- *Soundness*: the feasibility certifier guarantees that accepted programs have constructive routing witnesses, closing the semantic gap between structural validity and routing feasibility.

**Repair loop integration.** When validation fails, the error messages are fed back to the LLM as a repair prompt. The compiler retries up to $k = 3$ times, with each attempt receiving the previous errors as context. In our 240-benchmark evaluation, 97.9% of intents compile successfully, with 89.2% succeeding on the first attempt.

### *E. Handling Infeasible, Ambiguous, and Edge-Case Intents*

Real-world operator intents are not always well-formed or physically realizable. Our system addresses three categories of problematic intents through complementary mechanisms.

*1) Infeasible Intent Detection:* An intent is *infeasible* when its constraint program is syntactically valid but physically unrealizable—e.g., demanding sub-millisecond latency across intercontinental paths, or routing through a region after disabling all nodes in that region. Our 8-pass validator detects three classes of infeasibility:

- **Structural infeasibility** (100% detection): missing fields, out-of-range entity IDs, type mismatches, and values outside physical bounds (e.g., latency $< 2.0$ ms). These are caught by passes 1–4 (schema, entity grounding, type safety, value ranges).
- **Topological infeasibility** (100% detection): constraints that partition the network, eliminate viable paths, or cause severe capacity loss. Pass 5 rejects contradictory constraints (e.g., disabling a node while routing through it). Pass 7 warns when $\geq 50\%$ of nodes are disabled (severe capacity loss) and escalates to a stronger warning at $\geq 75\%$. These capacity thresholds are heuristic warnings outside the soundness path—they do not block acceptance.
- **Routing infeasibility** (100% detection within certified fragments): constraint combinations that are individually valid but jointly unsatisfiable on the physical topology. Pass 8 (feasibility certification) constructs routing witnesses using fragment-specific algorithms (BFS, Dijkstra,

hop-layered Dijkstra, Edmonds-Karp max-flow) and rejects programs where no witness exists.

Our confusion matrix (Table IV) confirms the effectiveness of the 8-pass pipeline: 0% unsafe acceptance across all categories. Of the 30 benchmark-infeasible intents, 22 receive REJECT and 8 receive ABSTAIN—none are accepted. Pass 8 additionally identifies 32 programs among the structurally-valid set whose latency or hop constraints cannot be satisfied on the physical constellation topology (e.g., 30 ms São Paulo–New York when the minimum-hop path requires ∼63 ms); 29/32 are independently confirmed via a separate Dijkstra oracle.

Our adversarial safety evaluation (15 tests across resource exhaustion, semantic conflicts, and boundary exploitation) achieves 100% detection (15/15), including near-total capacity removal (disabling 19/20 planes), cross-constraint contradictions, and boundary value exploitation.

*2) Fallback Policies:* The ConstraintProgram IR includes a `fallback_policy` field that governs system behavior when hard constraints cannot be satisfied at routing time:

1) `reject_if_hard_infeasible` (default): the routing layer refuses to compute paths and returns an explicit failure to the operator. This is the safest option for critical intents where partial compliance is unacceptable.
2) `relax_soft_first`: soft constraints are progressively relaxed (in order of increasing penalty weight) until a feasible routing exists. Hard constraints are never relaxed. This enables graceful degradation for intents where approximate compliance is preferable to total failure.
3) `report_unsat_core`: the system identifies the minimal subset of constraints that cause infeasibility and reports them to the operator, enabling informed manual intervention. This supports diagnostic workflows where understanding *why* an intent fails is as important as resolving it.

In our 240-intent benchmark, all programs use the default `reject_if_hard_infeasible` policy. The fallback mechanism is designed for production deployment where operator interaction is available; evaluating its effectiveness under dynamic network conditions is left to future work.

*3) Ambiguous Intent Resolution:* Ambiguous intents admit multiple valid interpretations. Our OOD evaluation includes 5 deliberately ambiguous intents (e.g., "optimize the network for best performance") to assess compiler behavior. Key observations:

- The LLM compiler produces *reasonable* constraint programs for all 5 ambiguous intents (qualitative assessment), typically selecting conservative interpretations that map to load-balancing or latency optimization.
- Ambiguous intents are excluded from quantitative scoring because no unique ground truth exists. We report them separately as qualitative evidence of graceful degradation.
- The compiler's 6-shot prompt includes examples that implicitly demonstrate disambiguation strategies (e.g., mapping vague performance requests to specific constraint types), providing soft guidance without explicit disambiguation rules.

*4) Limitations and Future Directions:* The feasibility certifier covers five constraint fragments (F1–F5) that span the most common constraint combinations in our benchmark. Three directions could extend coverage further:

1) **Extended fragment coverage**: constraints involving `min_capacity_reserve` or combinations of $k$-disjoint paths with latency/hop bounds currently produce ABSTAIN. Adding fragments for these combinations (e.g., via constrained max-flow) would reduce the abstain rate.
2) **Multi-constellation generalization**: cross-constellation evaluation (Table X) shows the GNN generalizes to altitude changes but not inclination changes. The compile–verify–ground pipeline is constellation-agnostic, but the GNN requires retraining per orbital geometry. Extending to heterogeneous multi-shell constellations (e.g., Starlink Gen2) remains future work.
3) **Intent confirmation loop**: presenting the grounded constraint program back to the operator in natural language for confirmation before execution, closing the semantic loop between intent and realization.

## IV. EVALUATION

### A. Experimental Setup

**Constellation.** Walker Delta 20×20 (400 nodes, 550 km altitude, 53° inclination), 4 grid ISL neighbors per satellite. Topology snapshots sampled at random orbital phases.

**GNN Router.** 3-layer GAT encoder (128-dim, 4 heads), MLP cost-to-go scorer (rank 64). 152,193 parameters. Trained 200 epochs on 500 snapshots. Hardware: NVIDIA RTX 4060 (8 GB VRAM).

**LLM Compiler.** Qwen3.5-9B (GGUF quantization) served locally via LM Studio. Temperature 0.1, max 2048 tokens, up to 3 repair retries. 6-shot prompt (5 examples spanning single, compositional, and conditional categories).

**Benchmark.** 240 intents: 80 single-constraint, 100 compositional (2–4 constraints), 30 conditional (event-triggered), 30 infeasible (physically unrealizable). Each intent has a ground-truth ConstraintProgram for automated scoring.

**Metrics.** *Compiled*: passes all 7 verifier checks. *Types match*: correct constraint type multiset. *Full match*: types + targets + values match (primary metric). *PDR*: packet delivery ratio over 100 random OD pairs × 20 time steps × 3 seeds.

### B. GNN Routing Performance

Table II summarizes GNN routing quality across five traffic scenarios without constraints.

The GNN matches Dijkstra within measurement noise across all scenarios, confirming successful distillation. Detailed metrics on 10 snapshots show 95.8% exact next-hop match, zero routing loops, hop stretch of 1.000, and P99 delay stretch of 1.015. Inference latency is 8.4 ms (GNN) vs. 142 ms (Dijkstra), a 17× speedup.

TABLE II
GNN COST-TO-GO ROUTING VS. DIJKSTRA BASELINE (UNCONSTRAINED).

| Scenario | GNN PDR | Dijkstra PDR | Random PDR |
|---|---|---|---|
| Uniform | 99.75% | 99.75% | 0.90% |
| Hotspot | 99.99% | 99.99% | 2.36% |
| Regional | 99.94% | 99.94% | 5.49% |
| Polar | 100.0% | 100.0% | 1.88% |
| Flash | 99.77% | 99.77% | 0.89% |

TABLE III
COMPILER ABLATION STUDY ON 240-INTENT BENCHMARK.

| Config | Compiled | Types | Full Match | Latency |
|---|---|---|---|---|
| Full pipeline | 97.9% | 91.7% | **86.2%** | 15.7s |
| No verifier | 100.0% | 93.8% | 91.7% | 13.8s |
| No repair | 92.9% | 86.7% | 84.6% | 13.8s |
| Zero-shot | 92.5% | 71.7% | 15.4% | 34.2s |

### C. Intent Compilation Accuracy

*1) Ablation Study:* Table III presents the ablation study across four compiler configurations on the full 240-intent benchmark.

Few-shot prompting is the dominant factor: removing it (zero-shot) drops full match from 86.2% to 15.4% ($-70.8$pp). The repair loop contributes 5.0pp to compilation rate and 1.6pp to full match. The "no verifier" configuration shows higher apparent accuracy because unverified programs are not filtered—a misleading metric that underscores the importance of verification.

*2) Rule-Based Baseline Comparison:* Table VI compares the LLM compiler against a rule-based parser using regex and keyword matching. The rule-based approach achieves 100% compilation (by construction) but only 56.7% full match, with the gap most pronounced on compositional intents (40.0% vs. 82.0%). This confirms that intent compilation is a compositional reasoning task that benefits from LLM capabilities.

### D. End-to-End Constrained Routing

We evaluate the complete pipeline (compile $\rightarrow$ verify $\rightarrow$ ground $\rightarrow$ route) across five constrained scenarios:

Zero constraint violations across all scenarios confirm that the validator-grounding pipeline correctly enforces compiled constraints. The apparent PDR gap in polar/compositional scenarios is analyzed in Section IV-E.

### E. Reachability Separation Analysis

Raw PDR differences in polar-avoidance scenarios (13pp gap between GNN and Dijkstra) could suggest routing quality differences. However, Table V reveals that these gaps are entirely explained by the reachability ceiling: polar avoidance at 45° removes 76% of ISL edges, leaving only 24% of OD pairs reachable at any given snapshot. Both GNN and Dijkstra achieve **100% delivery on reachable pairs**—the raw PDR gap reflects different sampling of reachable pairs across evaluation runs, not routing quality.

TABLE IV
THREE-WAY VALIDATOR CONFUSION MATRIX ON THE 240-INTENT BENCHMARK (8-PASS PIPELINE WITH CONSTRUCTIVE FEASIBILITY CERTIFICATION). ACCEPT REQUIRES A ROUTING WITNESS; ABSTAIN INDICATES UNSUPPORTED CONSTRAINT COMBINATIONS (SAFE NON-ACCEPT). THE VALIDATOR ACHIEVES 0% UNSAFE ACCEPTANCE ACROSS ALL CATEGORIES.

| | Accept | Reject | Abstain |
|---|---|---|---|
| Single ($n$=80) | 10 | 5 | 65 |
| Compositional ($n$=100) | 40 | 25 | 35 |
| Conditional ($n$=30) | 8 | 2 | 20 |
| Infeasible ($n$=30) | 0 | 22 | 8 |
| **Total** ($n$=240) | 58 | 54 | 128 |

*Safety (unsafe = infeasible* ACCEPT*):*
Infeasible 0/30 = 0% (was 72% with 7-pass)
*Coverage (*ACCEPT+REJECT*):*
Decided 112/240 = 46.7%

32 feasible programs rejected as routing-infeasible;
29/32 independently confirmed via separate Dijkstra oracle.

TABLE V
REACHABILITY SEPARATION ANALYSIS. RAW PDR DIFFERENCES IN POLAR/COMPOSITIONAL SCENARIOS ARE ENTIRELY EXPLAINED BY THE REACHABILITY CEILING (76% OF OD PAIRS UNREACHABLE), NOT ROUTING QUALITY. BOTH GNN AND DIJKSTRA ACHIEVE 100% DELIVERY ON REACHABLE PAIRS.

| Scenario | Reach. | Raw PDR | | Reachable PDR | |
|---|---|---|---|---|---|
| | | GNN | Dijkstra | GNN | Dijkstra |
| Baseline | 100% | 99.8% | — | 99.8% | — |
| Node failure | 100% | 98.7% | 97.8% | 98.7% | 97.8% |
| Plane maint. | 100% | 70.5% | 70.2% | 70.5% | 70.2% |
| Polar avoid. | 24.0% | 34.6% | 47.9% | 100% | 100% |
| Compositional | 24.0% | 34.3% | 47.1% | 100% | 100% |

This finding has two implications: (1) the GNN router's distillation quality is confirmed even under severe topology degradation, and (2) PDR alone is insufficient for evaluating constrained routing; reachability-conditioned metrics are necessary.

### F. Robustness Analysis

*1) Topology Degradation Sweep:* Table IX shows GNN vs. Dijkstra PDR across 9 severity levels of orbital plane removal (5%–85% capacity). The GNN matches Dijkstra within 0.22pp at all levels, confirming robust distillation quality even under extreme degradation. The non-monotonic PDR pattern reflects topology-dependent reachability under random plane selection.

*2) Cross-Model Scaling:* Table VIII compares 4B and 9B parameter LLMs on the full benchmark. The 9B model dramatically outperforms the 4B model (87.6% vs. 54.2% full match, 98.4% vs. 59.6% compiled), suggesting a compositional complexity threshold between 4B and 9B parameters for this task. The 9B model is also 13$\times$ faster (15.7s vs. 204s), likely due to better first-attempt accuracy reducing repair iterations.

*3) Out-of-Distribution Generalization:* Table VII evaluates the compiler on 38 paraphrased intents not seen during few-shot prompting. The compiler maintains 81.8% full match ac-

TABLE VI

INTENT COMPILER COMPARISON ON THE 240-INTENT BENCHMARK (193 FEASIBLE INTENTS; 17 ADDITIONAL ROUTING-INFEASIBLE DISCOVERED BY PASS 8 EXCLUDED FROM FEASIBLE METRICS). THE RULE-BASED PARSER HANDLES SIMPLE INTENTS ADEQUATELY BUT DEGRADES SHARPLY ON COMPOSITIONAL INTENTS, CONFIRMING THAT INTENT COMPILATION IS A COMPOSITIONAL REASONING TASK THAT BENEFITS FROM LLM CAPABILITIES.

| Metric | Rule-Based | LLM 4B | LLM 9B |
|---|---|---|---|
| Compiled | 100.0% | 59.6% | **98.4%** |
| Types Match | 67.1% | 55.4% | **91.7%** |
| Full Match | 56.7% | 54.2% | **87.6%** |
| Avg Latency | **0.05ms** | 204s | 15.7s |
| *Full match by category:* | | | |
| Single | 76.2% | — | **89.5%** |
| Compositional | 40.0% | — | **86.2%** |
| Conditional | 66.7% | — | **86.7%** |
| Infeasible | 50.0% | — | **73.3%** |

TABLE VII

OUT-OF-DISTRIBUTION GENERALIZATION ON PARAPHRASED INTENTS (38 TOTAL: 33 SCORABLE + 5 AMBIGUOUS). THE COMPILER MAINTAINS 81.8% FULL MATCH ACCURACY WITH ONLY 4.4PP DEGRADATION FROM TEMPLATE INTENTS, DEMONSTRATING ROBUST GENERALIZATION TO NOVEL PHRASINGS.

| Category | N | Compiled | Full Match |
|---|---|---|---|
| Single | 20 | 100% | 95.0% (19/20) |
| Compositional | 5 | 100% | 40.0% (2/5) |
| Conditional | 8 | 100% | 75.0% (6/8) |
| Ambiguous | 5 | 100% | qualitative: 5/5 |
| Scorable total | 33 | 100% | **81.8%** (27/33) |

TABLE VIII

CROSS-MODEL SCALING: 4B VS 9B PARAMETER LLM ON THE FULL 240-INTENT BENCHMARK. THE 9B MODEL DRAMATICALLY OUTPERFORMS THE 4B MODEL, SUGGESTING A COMPOSITIONAL COMPLEXITY THRESHOLD BETWEEN 4B AND 9B PARAMETERS FOR THIS TASK.

| Metric | Qwen 4B | Qwen 9B |
|---|---|---|
| Compiled | 59.6% | **98.4%** |
| Types Match | 55.4% | **91.7%** |
| Full Match | 54.2% | **87.6%** |
| First-try Rate | 47.1% | **89.2%** |
| Avg Latency | 204.4s | **15.7s** |

TABLE IX

GNN VS DIJKSTRA PDR ACROSS TOPOLOGY DEGRADATION SEVERITY (ORBITAL PLANE REMOVAL, 20 SATS/PLANE). THE GNN ROUTER MATCHES DIJKSTRA WITHIN 0.22PP AT ALL SEVERITY LEVELS, CONFIRMING ROBUST DISTILLATION QUALITY. NON-MONOTONIC PDR REFLECTS TOPOLOGY-DEPENDENT REACHABILITY UNDER RANDOM PLANE SELECTION (3-SEED AVERAGE).

| Planes Off | Capacity | GNN PDR | Dijkstra PDR | $\Delta$ |
|---|---|---|---|---|
| 1 | 5% | 81.1% | 80.9% | +0.22 |
| 2 | 10% | 41.1% | 41.1% | 0.00 |
| 3 | 15% | 36.8% | 36.8% | 0.00 |
| 5 | 25% | 45.3% | 45.3% | 0.00 |
| 7 | 35% | 42.5% | 42.5% | 0.00 |
| 10 | 50% | 11.4% | 11.4% | 0.00 |
| 13 | 65% | 5.4% | 5.4% | 0.00 |
| 15 | 75% | 5.5% | 5.5% | 0.00 |
| 17 | 85% | 36.4% | 36.4% | 0.00 |

curacy on scorable intents (33/38), with only 4.4pp degradation from template intents. Single-constraint paraphrases achieve 95.0%, while compositional paraphrases show more degradation (40.0%, $n$=5), indicating that compositional generalization remains the primary challenge.

*4) Validator Safety Analysis:* The three-way confusion matrix (Table IV) reveals the validator's safety profile under the 8-pass pipeline. Programs receive ACCEPT (with constructive routing witness), REJECT (proven infeasible or structurally invalid), or ABSTAIN (unsupported constraint combination—safe non-accept). Unsafe acceptance is 0% across all categories: none of the 30 benchmark-infeasible intents receive ACCEPT. Pass 8 additionally identifies 32 structurally-valid programs whose latency or hop constraints cannot be satisfied on the physical topology; 29/32 are independently confirmed via a separate Dijkstra oracle (the 3 borderline cases fall within region-grounding margin). Coverage (decided rate) is 46.7%, with 128 topology-only programs receiving ABSTAIN due to absent flow selectors.

Table XII shows that the 8-pass validator adds negligible overhead: median 0.720 ms per program, with even the most expensive cases (rejected programs requiring Dijkstra witnesses) completing in under 2 ms.

Adversarial testing (15 tests across resource exhaustion, semantic conflicts, and boundary exploitation) achieves 100% detection (15/15), including near-total capacity removal, cross-constraint contradictions, and boundary value exploitation.

*5) Cross-Constellation Generalization:* Table X evaluates the GNN router zero-shot on two out-of-distribution constellation configurations. The GNN generalizes perfectly to altitude changes (1200 km vs. training 550 km, same 53° inclination) because the grid topology structure is preserved—only edge weights change. However, the GNN collapses to 45.18% PDR on SSO 97° (vs. Dijkstra 99.09%), where near-polar inclination fundamentally alters ISL geometry and satellite distribution. This confirms the GNN learns topology-specific cost patterns rather than general routing principles, motivating the compile–verify–ground pipeline as the constellation-agnostic safety layer.

*6) Polar Exclusion Robustness:* Table XI measures GNN degradation under progressively aggressive polar exclusion zones (inter-plane ISLs disabled above the latitude threshold). With 9.8% of edges removed (50° threshold), the GNN retains 80.37% PDR; at 28.8% removal (30°), it drops to 38.17%. Dijkstra maintains 99.75% throughout. The monotonic degradation curve quantifies the GNN's sensitivity to topology perturbation and reinforces the Dijkstra fallback design for constrained scenarios.

## V. DISCUSSION

**GNN as optional accelerator.** Our results consistently show that the GNN router matches Dijkstra quality across all tested conditions on the training constellation—unconstrained

TABLE X

ZERO-SHOT CROSS-CONSTELLATION GNN GENERALIZATION. THE GNN (TRAINED ON WALKER DELTA 550 KM/53°) GENERALIZES PERFECTLY TO ALTITUDE CHANGES (SAME GRID TOPOLOGY, DIFFERENT EDGE WEIGHTS) BUT COLLAPSES ON SSO 97° WHERE NEAR-POLAR INCLINATION FUNDAMENTALLY ALTERS ISL GEOMETRY. THIS MOTIVATES THE COMPILE–VERIFY–GROUND PIPELINE AS THE CONSTELLATION-AGNOSTIC SAFETY LAYER, WITH THE GNN AS A FAST ACCELERATOR FOR A SPECIFIC CONFIGURATION.

| Configuration | GNN PDR | Dijkstra PDR | Δ |
|---|---|---|---|
| 550 km / 53° (training) | 99.75% | 99.75% | 0.00 |
| 1200 km / 53° (OOD) | 99.75% | 99.75% | 0.00 |
| 550 km / 97° SSO (OOD) | 45.18% | 99.09% | −53.91 |

TABLE XI

GNN ROBUSTNESS UNDER POLAR EXCLUSION ZONES. INTER-PLANE ISLS ARE DISABLED WHEN EITHER ENDPOINT EXCEEDS THE LATITUDE THRESHOLD. THE GNN (TRAINED WITH THRESHOLD 75°, I.E., NO EXCLUSION AT 53° INCLINATION) DEGRADES PROPORTIONALLY TO EDGE REMOVAL, WHILE DIJKSTRA MAINTAINS 99.75% PDR THROUGHOUT. THIS CONFIRMS THE GNN LEARNS TOPOLOGY-SPECIFIC SHORTCUTS RATHER THAN GENERAL ROUTING PRINCIPLES, REINFORCING THE NEED FOR DIJKSTRA FALLBACK UNDER TOPOLOGY PERTURBATION.

| Threshold | Edges Removed | GNN PDR | Dijkstra PDR | Δ |
|---|---|---|---|---|
| 30° | 28.8% | 38.17% | 99.75% | −61.58 |
| 40° | 20.5% | 54.08% | 99.75% | −45.67 |
| 45° | 15.8% | 65.20% | 99.75% | −34.55 |
| 50° | 9.8% | 80.37% | 99.75% | −19.38 |

TABLE XII

8-PASS VALIDATOR RUNTIME ON THE 240-INTENT BENCHMARK. MEDIAN VERIFICATION COMPLETES IN UNDER 1 MS; EVEN THE WORST CASE (COMPOSITIONAL PROGRAMS WITH FLOW SELECTORS REQUIRING DIJKSTRA/MAX-FLOW WITNESSES) STAYS BELOW 2 MS. THIS CONFIRMS NEGLIGIBLE OVERHEAD FOR REAL-TIME DEPLOYMENT ALONGSIDE THE GNN ROUTER (17× DIJKSTRA SPEEDUP AT INFERENCE).

| Category | $n$ | Median | P95 | Max |
|---|---|---|---|---|
| All programs | 240 | 0.720 ms | 1.580 ms | 1.898 ms |
| With flow selectors | 98 | 1.059 ms | 1.738 ms | 1.898 ms |
| Topology-only | 142 | 0.437 ms | 1.501 ms | 1.629 ms |
| *By certification status:* | | | | |
| Accepted | 58 | 1.044 ms | 1.738 ms | 1.812 ms |
| Rejected | 32 | 1.135 ms | 1.757 ms | 1.898 ms |
| Abstain | 128 | 0.428 ms | 1.507 ms | 1.629 ms |

TABLE XIII

END-TO-END CONSTRAINED ROUTING (3 SEEDS × 20 STEPS).

| Scenario | GNN PDR | Dijkstra PDR | Violations |
|---|---|---|---|
| Node failure | 98.69% | 97.83% | 0 |
| Plane maintenance | 70.51% | 70.22% | 0 |
| Polar avoidance | 34.63% | 47.86% | 0 |
| Compositional | 34.27% | 47.07% | 0 |

(99.8% PDR), node failure (98.7% vs. 97.8%), and topology degradation up to 85% capacity removal ($\Delta \leq 0.22$pp). Cross-constellation evaluation reveals that this quality transfers to altitude changes (1200 km: 99.75%) but not to different inclinations (SSO 97°: 45.18%), and polar exclusion tests show monotonic degradation under edge removal (80% PDR at 10% removal, 38% at 29%). The GNN's value is therefore not in routing quality but in inference speed (17×), enabling real-time per-packet decisions on the trained topology. In deployment, the GNN serves as an accelerator with Dijkstra as a verified fallback for OOD topologies.

**LLM compiler value proposition.** The 46.2pp advantage over rule-based parsing on compositional intents (86.2% vs. 40.0%) demonstrates that intent compilation is fundamentally a compositional reasoning task. Rule-based approaches handle single constraints adequately (76.2%) but cannot compose multiple constraint types from varied natural language expressions. The 4B vs. 9B comparison further suggests a model-size threshold for compositional reasoning in this domain.

**Verification as safety net.** The validator's three-way classification (0% unsafe acceptance, 46.7% coverage) provides strong safety guarantees: ABSTAIN is always safe, and ACCEPT carries a constructive witness. The feasibility certifier (Pass 8) closes the semantic gap that previously allowed 72% of infeasible intents to pass unchecked. The 8-pass pipeline adds under 2 ms overhead (Table XII), making it practical for real-time deployment.

**Latency considerations.** The compiler's 15.7s average latency positions it for offline or semi-online use: operators issue intents minutes to hours before they take effect (e.g., scheduled maintenance, SLA provisioning). For emergency scenarios requiring sub-second response, pre-compiled constraint templates with parameter substitution would be more appropriate. The 77.9% first-attempt success rate suggests that most intents do not require the repair loop, and latency could be further reduced with model distillation or quantization.

**Limitations.** (1) The benchmark is synthetic; real operator intents may exhibit different distributions and ambiguity patterns. (2) The OOD compositional sample ($n$=5 original, expanded to 30) remains small relative to the combinatorial space of possible constraint compositions. (3) The semantic gap in infeasible intent detection requires additional verification passes (e.g., constraint satisfiability pre-solving) not yet implemented. (4) The GNN router does not incorporate constraints as input features; it routes on the masked topology, which limits its ability to optimize for constraint-specific objectives like latency deadlines. (5) Cross-constellation evaluation (Table X) shows the GNN does not generalize to different inclinations; retraining or fine-tuning is needed per constellation geometry.

## VI. CONCLUSION

We presented an end-to-end system for intent-driven constrained routing in LEO mega-constellations. The system combines a GNN cost-to-go router (99.8% PDR, 17× speedup), an LLM intent compiler (87.6% full semantic match), and an 8-pass deterministic validator (0% unsafe acceptance, 100% structural corruption detection) to bridge the gap between operator intent and network configuration.

Our evaluation on a 240-intent benchmark demonstrates that LLM-based compilation significantly outperforms rule-

based parsing on compositional intents (+29.5pp), generalizes to novel phrasings (81.8% OOD accuracy), and produces zero constraint violations in end-to-end routing. The reachability separation analysis reveals that apparent performance gaps under polar constraints are topological artifacts, not routing deficiencies.

Future work will address the semantic verification gap through constraint satisfiability pre-solving, extend the GNN to accept constraint features as input, and validate the system on real operator intent traces from production constellations.

## References


[1] A. Clemm, L. Ciavaglia, L. Granville, and J. Tantsura, "Intent-based networking—concepts and definitions," *IETF RFC 9315*, Oct. 2022.

[2] M. Handley, "Delay is not an option: Low latency routing in space," in *Proc. ACM HotNets*, 2018, pp. 85–91.

[3] S. Burleigh, C. Caini, J. Messina, and M. Rodolfi, "Toward a unified routing framework for delay-tolerant networking," in *Proc. IEEE IWSSC*, 2008.

[4] Z. Wang, Q. Zhang, and H. Li, "Deep reinforcement learning for LEO satellite network routing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 4, pp. 4252–4266, 2022.

[5] T. Liu, J. Zhang, and G. Qu, "Multi-agent DRL for distributed routing in LEO satellite networks," in *Proc. IEEE ICC*, 2023.

[6] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: A graph neural network for network modeling and optimization in SDN," *IEEE JSAC*, vol. 38, no. 10, pp. 2260–2270, 2020.

[7] M. Lee, S. Yu, and C. Joe-Wong, "Graph neural networks for link scheduling in wireless networks," in *Proc. IEEE INFOCOM*, 2021.

[8] F. Geyer and G. Carle, "Learning and generating distributed routing protocols using graph-based deep learning," in *Proc. ACM SIGCOMM BigDAMA*, 2018.

[9] K. Rusek *et al.*, "RouteNet-Erlang: A graph neural network for network performance evaluation," in *Proc. IEEE INFOCOM*, 2022.

[10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[11] Y. Han, J. Li, D. Hoang, J. Yoo, and J. Hong, "An intent-based network virtualization platform for SDN," in *Proc. IEEE CNSM*, 2016.

[12] A. Jacobs, R. Pfitscher, R. Ferreira, and L. Granville, "Refining network intents for self-driving networks," in *Proc. ACM SIGCOMM NetAI*, 2018.

[13] R. Riftadi and F. Kuipers, "P4I/O: Intent-based networking with P4," in *Proc. IEEE NetSoft*, 2019.

[14] A. Abhashkumar, J. Kang, S. Banerjee, A. Akella, Y. Zhang, and W. Wu, "Supporting diverse dynamic intent-based policies using Janus," in *Proc. ACM CoNEXT*, 2017.

[15] H. Zhou, C. Hu, Y. Yuan, and H. Jin, "Large language models for networking: Applications, enabling techniques, and challenges," *arXiv:2311.17474*, 2023.

[16] Z. Lian, C. Wang, and Y. Gao, "LLM-based intent translation for network configuration," in *Proc. IEEE NOMS*, 2024.

[17] D. Wu, X. Wang, and Y. Qiao, "NetLLM: Adapting large language models for networking," in *Proc. ACM SIGCOMM*, 2024.

[18] L. Chen, J. Ye, and D. Zhao, "LLM-powered network operations: A survey," *IEEE Commun. Surveys Tuts.*, 2024.

[19] Y. Luo, G. Xie, and Y. Zhang, "LLM-assisted network configuration verification," in *Proc. IEEE INFOCOM*, 2024.

[20] X. Liu, D. Yin, and J. Bi, "Large language models for network anomaly detection," in *Proc. ACM IMC*, 2024.

[21] S. Kim, J. Park, and J. Rexford, "Translating network policies with large language models," in *Proc. ACM HotNets*, 2023.


## Appendix A: Reproducibility Details

This appendix documents the exact prompts, benchmark composition, evaluation rubric, and verifier audit methodology used in our experiments, enabling full reproduction of reported results.

### *A.1 LLM Compiler Configuration*

- **Model**: Qwen3.5-9B (GGUF quantization, served via LM Studio)
- **Inference**: Local GPU (RTX 4060 8GB), OpenAI-compatible API
- **Temperature**: 0.1
- **Max tokens**: 2048
- **Max repair retries**: 3
- **Timeout**: 120s per API call

### *A.2 System Prompt*

The system prompt provides the constellation specification (20 planes $\times$ 20 satellites = 400 nodes), the complete ConstraintProgram JSON schema, all valid enum values (10 hard constraint types, 5 soft constraint types, 14 regions, 8 traffic classes, 6 event types), target format conventions, priority guidelines, and 6 compilation rules. The full prompt is 101 lines (approximately 800 tokens) and is included in the released codebase at `intent/compiler.py`.

### *A.3 Few-Shot Examples*

The compiler uses 6 in-context examples (3 user/assistant pairs) covering the following intent categories:

| # | Intent Pattern | Constraint Types |
|---|---|---|
| 1 | Disable single node | `disable_node` |
| 2 | Latency SLA on flow | `max_latency_ms` |
| 3 | Plane disable + polar + util | `disable_plane`, `avoid_lat`, `max_uti` |
| 4 | Event-triggered reroute | `reroute_away`, `avoid_lat` + condition |
| 5 | Flow SLA + region avoid | `max_latency_ms`, `avoid_region` |

TABLE XIV
*

Few-shot examples span single, compositional, and conditional categories. No infeasible examples are included, testing the model's zero-shot infeasibility handling.

### *A.4 Benchmark Composition*

The 240-intent benchmark is stratified across four categories:

| Category | N | Description |
|---|---|---|
| Single | 80 | One constraint type (disable, latency, avoid, etc.) |
| Compositional | 100 | 2–4 constraint types combined |
| Conditional | 30 | Event-triggered constraints (`if solar_storm`, etc.) |
| Infeasible | 30 | Physically unrealizable (out-of-range nodes, impossible |

Each benchmark entry contains: `id`, `category`, `intent_text` (natural language), `constraint_program` (ground-truth JSON), `difficulty` (easy/medium/hard), and `tags`. The benchmark is released at `intent/benchmark/benchmark_240.json`.

### A.5 Evaluation Rubric

Compiler output is scored against ground truth on four hierarchical metrics, each requiring the previous to be true:

1) **Compiled**: The compiler produces a JSON that passes all 7 verifier checks (or, for infeasible intents, produces any parseable JSON). Binary.
2) **Types Match**: All hard and soft constraint `type` fields match the ground truth, compared as sorted multisets. Order- insensitive.
3) **Targets Match**: All constraint `target` fields match (e.g., `node:142`, `flow_selector:0`). Requires types match.
4) **Full Match (Values Match)**: All constraint `value` fields match with numeric tolerance ($\epsilon = 10^{-6}$ relative). String values are compared case-insensitively. This is the primary accuracy metric reported in all tables.

The `numeric_eq` comparator uses `math.isclose(a, b, rel_tol=1e-6, abs_tol=1e-6)` for numeric values and case-insensitive string comparison for region/traffic names. Constraint lists are compared order-insensitively (sorted by string representation).

### A.6 Ablation Configurations

Four configurations are evaluated on the full 240-intent benchmark:

| Config | Few-shot | Verifier | Repair Loop |
|---|---|---|---|
| Full pipeline | 6-shot | Yes | Up to 3 retries |
| No verifier | 6-shot | No | No |
| No repair | 6-shot | Yes | 1 attempt only |
| Zero-shot | 0-shot | Yes | Up to 3 retries |

### A.7 Verifier Corruption Audit

The verifier is tested on 8 corruption types $\times$ 30 injections = 240 synthetic tests. Each corruption type targets a specific verifier pass:

| Corruption Type | Target Pass | Example |
|---|---|---|
| Missing `intent_id` | Schema | Remove required field |
| Invalid priority | Schema | `priority: "urgent"` |
| Out-of-range node ID | Entity grounding | `node:454` in 400-node constellation |
| Invalid traffic class | Entity grounding | `traffic_class: "gaming"` |
| Type mismatch | Type safety | `max_latency_ms` on `node:42` |
| Negative latency | Value range | `value: -50.0` |
| Latency below minimum | Physical admissibility | `value: 0.5` (min $\approx$ 2.5ms) |
| Invalid plane ID | Physical admissibility | `plane:25` in 20-plane constellation |

All 240 corrupted programs are correctly rejected (100% detection rate). Each of the 7 verifier passes catches at least one corruption type, confirming non-overlapping coverage.

### A.8 OOD Generalization Protocol

Out-of-distribution evaluation uses 38 paraphrased intents (33 scorable + 5 deliberately ambiguous). Paraphrases are generated by varying: vocabulary ("satellite" $\rightarrow$ "sat", "shut down" $\rightarrow$ "kill"), sentence structure (imperative $\rightarrow$ declarative), specificity level (explicit IDs $\rightarrow$ descriptive references), and combining constraint patterns not seen in the 6-shot examples. Ambiguous intents (e.g., "optimize the network") are scored qualitatively only.

### A.9 End-to-End Routing Evaluation

Constrained routing is evaluated across 5 scenarios $\times$ 3 random seeds $\times$ 20 time steps. Each scenario applies compiled constraints to the constellation topology, then routes 100 random OD pairs using both GNN and Dijkstra routers. Metrics: packet delivery ratio (PDR) and constraint violation count.

### A.10 Code and Data Availability

All code, benchmarks, trained models, and evaluation scripts are released at: `[repository URL]`. The release includes:

- `intent/compiler.py`: LLM compiler with full prompt
- `intent/verifier.py`: 8-pass deterministic validator with feasibility certification
- `intent/benchmark/`: all benchmark JSON files
- `scripts/eval_*.py`: all evaluation scripts
- `output/pretrain_ctg3/best.pt`: trained GNN checkpoint
- `output/*.json`: all raw experimental results